

Практическая работа.

Работа с данными различных форматов: Excel, CSV, JSON, XML, HTML.

Цель работы:

Освоить практические навыки работы с данными в различных форматах (CSV, Excel, JSON, XML, HTML), включая чтение, обработку, трансформацию и запись данных с использованием библиотеки Pandas.

Задачи работы:

1. Освоить методы чтения и записи данных в форматах CSV, Excel, JSON, XML, HTML.
2. Научиться выполнять базовые операции по очистке и трансформации данных.
3. Понять особенности и ограничения различных форматов данных.
4. Освоить методы конвертации данных между различными форматами.
5. Применить полученные навыки для решения прикладных задач.

Теоретическая часть

Форматы данных в инжиниринге данных.

В современном инжиниринге данных работа с различными форматами является критически важным навыком [1]. Каждый формат имеет свои преимущества, недостатки и оптимальные области применения [2].

Формат	Преимущества	Недостатки	Применение
CSV	Простота, универсальность, читаемость	Большой размер, отсутствие типов данных	Экспорт данных, малые датасеты
Excel	Визуализация, форматирование, формулы	Ограничения по размеру, сложность обработки	Бизнес-отчеты, аналитика
JSON	Гибкость, поддержка вложенности, работа с API	Избыточность, медленная обработка больших данных	Веб-API, логи, конфигурации
XML	Схемы валидации, иерархичность	Многословность, сложность парсинга	Корпоративные системы, обмен данными

Таблица 1: Сравнение форматов данных.

Библиотека Pandas для работы с данными.

Pandas предоставляет унифицированный интерфейс для работы с различными форматами данных через объект DataFrame. Основные методы чтения и записи данных представлены в таблице ниже.

Формат	Метод чтения	Метод записи
CSV	pd.read_csv()	df.to_csv()
Excel	pd.read_excel()	df.to_excel()
JSON	pd.read_json()	df.to_json()
XML	pd.read_xml()	df.to_xml()

Table 2: Методы Pandas для работы с форматами данных

Подготовка к работе.

Необходимое программное обеспечение.

- Python 3.8 или выше
- Jupyter Notebook или JupyterLab
- Библиотеки: pandas, openpyxl, lxml

Установка библиотек.

```
pip install pandas openpyxl lxml xlrd
```

Создание тестового набора данных.

Создайте директорию для работы и подготовьте следующую структуру:

```
data_formats_lab/  
├── data/  
│   ├── raw/  
│   ├── processed/  
│   └── output/  
├── notebooks/  
└── lab_work.ipynb
```

1. Работа с CSV-файлами.

Задача 1.1: Чтение CSV-файла

Создайте CSV-файл. (Например, sales_data.csv с содержимым следующей структуры:

```
date,product,category,price,quantity,region
2024-01-15,Laptop,Electronics,45000,2,Moscow
2024-01-16,Mouse,Accessories,800,5,St. Petersburg
2024-01-16,Keyboard,Accessories,1500,3,Moscow
2024-01-17,Monitor,Electronics,12000,1,Kazan
2024-01-18,Laptop,Electronics,48000,1,Moscow
2024-01-19,USB Cable,Accessories,300,10,St. Petersburg
...)
```

Можно выбрать свой набор данных.

Напишите код для чтения этого файла.

Выведите первые 5 строк.

Выведите информацию о данных.

Выведите базовые статистические характеристики (среднее значение, мода, медиана, ...)

Обработка и трансформация данных.

Выполните следующие операции:

Конвертация столбца date в datetime

```
df['date'] = pd.to_datetime(df['date'])
```

Создание нового столбца total_amount

```
df['total_amount'] = df['price'] * df['quantity']
```

Группировка по категориям

```
category_sales = df.groupby('category')['total_amount'].sum()
print("\nПродажи по категориям:")
print(category_sales)
```

Фильтрация данных (продажи > 10000)

```
high_value_sales = df[df['total_amount'] > 10000]
print("\nКрупные продажи:")
print(high_value_sales)
```

Запись результатов в CSV.

Сохранение обработанных данных

```
df.to_csv('data/processed/sales_processed.csv', index=False)
```

Сохранение агрегированных данных

```
category_sales.to_csv('data/output/category_summary.csv')
```

Сохранение с другим разделителем (TSV)

```
df.to_csv('data/output/sales_data.tsv', sep='\t', index=False)
```

Вопросы для самоконтроля

1. Какой параметр используется для указания разделителя в CSV?
2. Как обработать CSV-файл с кириллическими символами?
3. Что делает параметр `index=False` при сохранении?

2: Работа с Excel-файлами.

2.1: Создание и запись Excel-файла.

```
import pandas as pd
```

Создание нескольких DataFrame для разных листов.

```
sales_2023 = pd.DataFrame({  
'month': ['January', 'February', 'March'],  
'revenue': [150000, 180000, 165000],  
'costs': [90000, 100000, 95000]  
})
```

```
sales_2024 = pd.DataFrame({  
'month': ['January', 'February', 'March'],  
'revenue': [175000, 190000, 185000],  
'costs': [95000, 105000, 100000]  
})
```

Запись в Excel с несколькими листами.

```
with pd.ExcelWriter('data/output/annual_report.xlsx',  
engine='openpyxl') as writer:  
sales_2023.to_excel(writer, sheet_name='2023', index=False)  
sales_2024.to_excel(writer, sheet_name='2024', index=False)
```

2.2: Чтение Excel-файлов.

Чтение конкретного листа.

```
df_2023 = pd.read_excel('data/output/annual_report.xlsx',  
sheet_name='2023')  
print("Данные за 2023 год:")  
print(df_2023)
```

Чтение всех листов.

```
all_sheets = pd.read_excel('data/output/annual_report.xlsx',  
sheet_name=None)  
for sheet_name, data in all_sheets.items():  
print(f"\nЛист: {sheet_name}")  
print(data)
```

Чтение диапазона ячеек.

```
df_range = pd.read_excel('data/output/annual_report.xlsx',  
sheet_name='2023',  
usecols='A:B',  
nrows=2)
```

```
print("\nВыборочное чтение:")  
print(df_range)
```

2.3. Продвинутая работа с Excel.

Создание сводной таблицы.

```
pivot_data = pd.DataFrame({  
    'date': ['2024-01'] * 3 + ['2024-02'] * 3,  
    'product': ['A', 'B', 'C'] * 2,  
    'sales': [100, 150, 200, 120, 160, 210]  
})
```

```
pivot_table = pivot_data.pivot_table(  
    values='sales',  
    index='product',  
    columns='date',  
    aggfunc='sum'  
)
```

Сохранение с форматированием.

```
with pd.ExcelWriter('data/output/pivot_report.xlsx',  
    engine='openpyxl') as writer:  
    pivot_table.to_excel(writer, sheet_name='Pivot')
```

Доступ к workbook для дополнительного форматирования

```
workbook = writer.book  
worksheet = writer.sheets['Pivot']
```

добавить дополнительное форматирование здесь

Вопросы для самоконтроля

1. Какой engine используется по умолчанию для записи Excel?
2. Как прочитать только определенные столбцы из Excel?
3. В чем разница между `openpyxl` и `xlrd`?

3: Работа с JSON-файлами.

Задача 3.1: Простой JSON.

```
import pandas as pd
import json
```

Создание простого JSON.

```
simple_data = {
    'employees': [
        {'id': 1, 'name': 'Ivanov', 'department': 'IT', 'salary': 80000},
        {'id': 2, 'name': 'Petrov', 'department': 'HR', 'salary': 65000},
        {'id': 3, 'name': 'Sidorov', 'department': 'IT', 'salary': 90000}
    ]
}
```

Сохранение в JSON.

```
with open('data/raw/employees.json', 'w', encoding='utf-8') as f:
    json.dump(simple_data, f, ensure_ascii=False, indent=2)
```

Чтение JSON с pandas.

```
df = pd.read_json('data/raw/employees.json')
print(df)
```

3.2. Вложенный JSON.

Пример вложенной структуры

```
nested_json = """
{
    "company": "TechCorp",
    "employees": [
        {
            "id": 1,
            "name": "Ivanov",
            "contact": {
                "email": "ivanov@example.com",
                "phone": "+7-999-123-4567"
            },
            "projects": ["Project A", "Project B"]
        },
        {
            "id": 2,
            "name": "Petrov",
```



```
"contact": {
  "email": "petrov@example.com",
  "phone": "+7-999-765-4321"
},
"projects": ["Project C"]
}
]
```

Сохранение.

```
with open('data/raw/company.json', 'w', encoding='utf-8') as f:
    f.write(nested_json)
```

Чтение с нормализацией.

```
import json
with open('data/raw/company.json', 'r', encoding='utf-8') as f:
    data = json.load(f)
```

Нормализация вложенной структуры.

```
df_normalized = pd.json_normalize(
    data['employees'],
    meta=['id', 'name'],
    record_path='projects',
    record_prefix='project_'
)
print("\nНормализованные данные:")
print(df_normalized)
```

3.3. Различные ориентации JSON.

Создание DataFrame

```
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': [25, 30, 35],
    'city': ['Moscow', 'SPb', 'Kazan']
})
```

Сохранение в разных форматах.

```
df.to_json('data/output/records.json', orient='records', force_ascii=False)
df.to_json('data/output/index.json', orient='index', force_ascii=False)
```

```
df.to_json('data/output/columns.json', orient='columns', force_ascii=False)
df.to_json('data/output/values.json', orient='values', force_ascii=False)
```

Чтение и сравнение.

```
print("Records format:")
print(pd.read_json('data/output/records.json'))

print("\nColumns format:")
print(pd.read_json('data/output/columns.json'))
```

Вопросы для самоконтроля

1. Какие параметры orient доступны в to_json()?
2. Что делает функция pd.json_normalize()?
3. Как обработать JSON с нестандартной кодировкой?

4: Работа с XML-файлами.

4.1. Простой XML.

```
import pandas as pd
```

Создание простого XML.

```
xml_data = """McKinney 2022 1500Vanderplas 2023 1200Smith 2024 1800
"""
```

Сохранение XML.

```
with open('data/raw/catalog.xml', 'w', encoding='utf-8') as f:
    f.write(xml_data)
```

Чтение XML.

```
df = pd.read_xml('data/raw/catalog.xml', xpath='//book')
print(df)
```

Базовая обработка.

```
df['price'] = df['price'].astype(int)
avg_price = df['price'].mean()
print(f"Средняя цена книг: {avg_price:.2f}")
```

4.2. XML с атрибутами.

XML с атрибутами.

```
xml_with_attrs = """ Laptop 45000 15 Mouse 800 50 Monitor 12000 8
"""
```

```
with open('data/raw/products.xml', 'w', encoding='utf-8') as f:
    f.write(xml_with_attrs)
```

Чтение с атрибутами.

```
df = pd.read_xml('data/raw/products.xml')
print(df)
```

Запись обратно в XML.

```
df.to_xml('data/output/products_modified.xml',
index=False,
root_name='products',
row_name='product')
```

4.3: Парсинг сложного XML с помощью lxml.

```
from lxml import etree
```

Чтение XML для более сложной обработки.

```
tree = etree.parse('data/raw/products.xml')
root = tree.getroot()
```

Извлечение данных с помощью XPath.

```
products = []
for product in root.xpath('//product'):
    products.append({
        'id': product.get('id'),
        'category': product.get('category'),
        'name': product.find('name').text,
        'price': int(product.find('price').text),
        'currency': product.find('price').get('currency'),
        'stock': int(product.find('stock').text)
    })

df = pd.DataFrame(products)
print(df)
```

Фильтрация и анализ.

```
electronics = df[df['category'] == 'electronics']
print(f"\nОбщая стоимость электроники на складе: {(electronics['price'] *
electronics['stock']).sum()}")
```

Вопросы для самоконтроля.

1. Какой параметр используется для указания XPath при чтении XML?
2. Чем отличается `pd.read_xml()` от прямого парсинга с lxml?
3. Как обработать XML с пространствами имен?

5. Интеграция форматов.

5.1. Конвертация между форматами.

Создайте класс для конвертации данных между всеми форматами:

```
import pandas as pd

class DataConverter:
    """Класс для конвертации данных между форматами"""

    def __init__(self, input_path):
        ...

    def read_data(self, file_format):
        """Чтение данных в зависимости от формата"""
        ...

    def convert_to_all(self, output_dir):
        """Конвертация в все форматы"""

        # CSV
        ...

        # Excel
        ...

        # JSON
        ...

        # XML
        ...
```

Использование конвертера.

...

5.2. ETL-процесс с множественными источниками.

Написать класс для агрегации данных из разных источников в один датасет.

```
import pandas as pd
import os

def etl_process():
    """Полный ETL-процесс с данными из разных источников"""
```

```
# Extract: загрузка из разных источников
print("\n=== EXTRACT PHASE ===")

# CSV: данные о продажах

# JSON: данные о сотрудниках

# XML: данные о продуктах

# Transform: обработка и объединение
print("\n=== TRANSFORM PHASE ===")

# Агрегация продаж

# Добавление информации о средней цене

print("Данные агрегированы")

# Load: сохранение результатов
print("\n=== LOAD PHASE ===")

# Создание директории для результатов

# Сохранение в разных форматах
```

Запуск ETL-процесса

5.3. Бенчмаркинг форматов.

Написать класс для сравнения производительности различных форматов.

Для этого сгенерировать наборы данных, содержащие различное количество записей.

Визуализировать скорость чтения и записи данных в различных форматах для различных объемов данных.

6: Практическое применение.

Индивидуальное задание.

Разработайте систему обработки данных для следующего сценария:

Сценарий: Вы работаете дата-инженером в компании электронной коммерции. Вам необходимо:

1. Загрузить данные о заказах из CSV-файла.
2. Загрузить информацию о продуктах из Excel-файла.
3. Загрузить данные о клиентах из JSON.
4. Объединить данные из всех источников
5. Выполнить аналитику (топ-10 продуктов, распределение по регионам)
6. Сохранить результаты в формате Excel с несколькими листами
7. Создать JSON-отчет для веб-дашборда.

Контрольные вопросы

1. Какие преимущества CSV перед Excel для хранения больших объемов данных?
2. Когда следует использовать JSON вместо CSV?
3. Что такое ориентация JSON и какие варианты существуют?
4. Как обработать Excel-файл с несколькими листами?
5. Какие проблемы могут возникнуть при работе с XML-данными?
6. Чем отличается `read_csv()` от `read_table()`?
7. Как настроить кодировку при чтении файлов?
8. Что такое нормализация JSON и когда она необходима?
9. Какие параметры влияют на производительность при работе с большими файлами?
10. Как выбрать оптимальный формат для конкретной задачи?

Полезные ссылки

- Официальная документация Pandas: <https://pandas.pydata.org/docs/>
- Pandas Cheat Sheet: https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf
- Real Python - Working with Files: <https://realpython.com/working-with-files-in-python/>